

Open-World Knowledge Retrieval Architecture

1 Open-World Knowledge Retrieval Architecture

The proposed system implements an open-world knowledge retrieval pipeline that combines query analysis, data-derived schema-value resolution, dynamically learned query canonicalization, metadata-aware hybrid retrieval, reciprocal rank fusion, and relevance reranking.

The complete query-time pipeline is:

$$\boxed{\text{User Query} \rightarrow \text{Query Analysis} \rightarrow \text{Canonicalization} \rightarrow \text{Cache Lookup} \rightarrow \text{Hybrid Retrieval} \rightarrow \text{RRF} \rightarrow \text{Candidate Ranking}} \quad (1)$$

The implementation is designed around scalable retrieval primitives, while the current reference implementation deliberately retains several development and validation mechanisms. In particular, the dense HNSW index supports incremental insertion, whereas the current `rank_bm25` implementation uses deferred full-corpus rebuilding. Similarly, brute-force dense retrieval is used only for ANN recall validation and is not part of the production retrieval path.

The architecture should therefore be interpreted as a scalable retrieval design with a reference implementation, rather than as a claim that every individual component is already suitable for large-scale production deployment.

1.1 System Overview

The architecture contains six principal logical stages:

1. **Query analysis:** converts the input query into an embedding, extracts a closed set of structured comparison constraints, and attempts to resolve query values against metadata values observed in the knowledge base.
2. **Dynamic canonicalization:** compares the query embedding with the current set of dynamically created query-cluster centroids. A query joins the highest-scoring existing cluster when the similarity exceeds a configurable threshold; otherwise, a new cluster is created.
3. **Canonical cache:** constructs a cache key from the canonical query representation and tenant context. A cache hit allows the previously stored retrieval result to be returned without executing the retrieval layer again.
4. **Hybrid retrieval:** independently retrieves candidates using dense approximate nearest-neighbor search over an HNSW index and sparse BM25 retrieval.
5. **Rank fusion and reranking:** combines the dense and sparse rankings using Reciprocal Rank Fusion (RRF), after which a smaller candidate set is passed to an external relevance reranker.
6. **Evidence selection:** retains the highest-scoring reranked passages as evidence for the downstream answer-generation stage.

1.2 End-to-End Query Workflow

Figure 1.2 summarizes the query-time workflow.

End-to-end query workflow. Query analysis produces an embedding and extracts structured constraints. Schema-value resolution and dynamic clustering contribute structured metadata constraints and a canonical query representation. The resulting canonical key is checked against a tenant-aware cache. On a cache miss, dense HNSW and sparse BM25 retrieval generate candidate rankings. These rankings are fused using Reciprocal Rank Fusion (RRF), after which the fused candidate set is passed to an external relevance reranker. The highest-scoring passages are retained as evidence for answer generation.

1.3 Query Analysis

Given an input query q , the system first computes an embedding:

$$\mathbf{v}_q = f(q), \quad (2)$$

where $f(\cdot)$ denotes the embedding function.

In the current reference implementation, the embedding function uses the vector representation produced by spaCy’s `en_core_web_md` model. The embedding function is treated as a replaceable component so that a production deployment can substitute a sentence-embedding model or an external embedding service.

The resulting vector is normalized when its norm is non-zero:

$$\hat{\mathbf{v}}_q = \begin{cases} \frac{\mathbf{v}_q}{\|\mathbf{v}_q\|_2}, & \text{if } \|\mathbf{v}_q\|_2 > 0, \\ \mathbf{v}_q, & \text{otherwise.} \end{cases} \quad (3)$$

The query-analysis stage also extracts a small closed set of structured comparison expressions. The current implementation supports expressions corresponding to:

- “over” and “more than”,
- “under” and “less than”, and
- “last N days”.

For example, the query

“show debit transactions over \$100 from the last 30 days”

can produce structured slots such as

$$\mathcal{S}(q) = \{\text{amount_gt} = 100, \text{days} = 30\}. \quad (4)$$

These structured constraints are incorporated into the canonical query representation and remain available as structured query information during retrieval.

1.4 Data-Derived Schema-Value Resolution

The system does not require a manually authored linguistic alias list for every domain value. Instead, candidate metadata values are derived directly from the documents in the knowledge base.

For example, if the knowledge base contains the metadata field `transaction_type`, its observed values may include

$$\mathcal{V}_{\text{transaction_type}} = \{\text{DEBIT}, \text{CREDIT}\}. \quad (5)$$

Similarly, a **status** field may contain

$$\mathcal{V}_{\text{status}} = \{\text{PENDING}, \text{COMPLETED}\}. \quad (6)$$

These values are therefore discovered from the knowledge base rather than being specified in a manually maintained linguistic registry.

For each distinct schema value s , the system computes an embedding:

$$\mathbf{v}_s = f(s). \quad (7)$$

When normalized embeddings are used, similarity between the query and a schema value is computed as

$$S(q, s) = \hat{\mathbf{v}}_q^\top \hat{\mathbf{v}}_s. \quad (8)$$

The best candidate schema value s^* is accepted only when its similarity exceeds the similarity of the best background anchor by a configured margin:

$$S(q, s^*) \geq S(q, b^*) + \delta, \quad (9)$$

where b^* denotes the most similar background anchor and δ denotes the configured schema-resolution margin.

This approach removes the requirement for a manually maintained alias list such as

debit $\rightarrow$ withdrawal $\rightarrow$ outgoing payment $\rightarrow$ money taken out $\rightarrow \dots$

However, matching a query directly against a bare schema value is weaker than matching against a curated alias set or example-based representation. The architecture therefore reduces manual domain configuration at the cost of greater dependence on embedding quality and the quality of the observed metadata values.

1.5 Dynamic Canonical Query Registry

The **AutoCanonicalRegistry** dynamically creates query clusters as queries arrive.

Let the registry contain m clusters:

$$\mathcal{C} = \{c_1, c_2, \dots, c_m\}. \quad (10)$$

Each cluster c_i contains, at minimum,

- a cluster identifier,
- a centroid $\boldsymbol{\mu}_i$,
- a representative query,
- a query count, and
- associated metadata constraints when applicable.

For an incoming query embedding $\hat{\mathbf{v}}_q$, the system computes its similarity to each current cluster centroid:

$$S_i(q) = \text{sim}(\hat{\mathbf{v}}_q, \boldsymbol{\mu}_i). \quad (11)$$

The highest-scoring cluster is selected:

$$i^* = \arg \max_{i \in \{1, \dots, m\}} S_i(q). \quad (12)$$

If

$$S_{i^*}(q) \geq \tau, \quad (13)$$

where τ is the configured cluster-join threshold, the query joins cluster c_{i^*} .

If no existing cluster satisfies the threshold, a new cluster is created.

The representative query associated with the selected cluster is then used as the canonical retrieval representation.

1.6 Canonicalization and Arrival-Order Dependence

The clustering mechanism should not be interpreted as a mathematically guaranteed semantic-equivalence relation.

When a query joins an existing cluster, the centroid is updated using a running mean. If the cluster currently contains n_i query embeddings, the update is

$$\boldsymbol{\mu}'_i = \frac{n_i \boldsymbol{\mu}_i + \hat{\mathbf{v}}_q}{n_i + 1}. \quad (14)$$

It is important to distinguish two properties.

First, for a fixed set of cluster members, the arithmetic mean itself is independent of the order in which those members are averaged. Therefore, the running-mean update does not by itself introduce order dependence into the final centroid for an identical set of members.

Second, and more importantly, cluster *membership decisions* are made sequentially. A query arriving at time t is evaluated against the centroids produced by the queries that arrived before it. Consequently, different arrival orders can lead to different membership decisions and therefore to different final cluster compositions.

For example, consider three semantically related queries

$$q_1, \quad q_2, \quad q_3. \quad (15)$$

The sequence

$$q_1 \rightarrow q_2 \rightarrow q_3 \quad (16)$$

can produce different cluster assignments from

$$q_3 \rightarrow q_2 \rightarrow q_1, \quad (17)$$

because the centroid against which each subsequent query is evaluated can differ.

Thus,

$$q_a \equiv_{\text{semantic}} q_b \not\Rightarrow \text{cluster}(q_a) = \text{cluster}(q_b). \quad (18)$$

The system therefore does not guarantee that arbitrary semantically equivalent paraphrases will collapse to the same canonical cluster.

The narrower guarantee is:

$$\text{cluster}(q_a) = \text{cluster}(q_b) \Rightarrow \text{canonical_text}(q_a) = \text{canonical_text}(q_b), \quad (19)$$

provided that the cluster representative remains unchanged.

Cluster membership therefore provides consistent canonical treatment after assignment, but does not constitute a universal semantic equivalence relation.

1.7 Canonical Query Construction

The canonical query combines the selected cluster representation with any structured query constraints that affect retrieval.

Let $r(c(q))$ denote the representative text of the cluster assigned to query q , and let $\mathcal{S}(q)$ denote its structured slots. The canonical retrieval representation can then be written abstractly as

$$Q_{\text{canon}}(q) = r(c(q)) \oplus \text{serialize}(\mathcal{S}(q)), \quad (20)$$

where $\oplus$ denotes concatenation and $\text{serialize}(\cdot)$ converts structured constraints into a retrieval-compatible representation.

For example, if the cluster representative is

“give me all debit transactions”

and the incoming query contains the constraint “over \$100”, the canonical retrieval text may become

“give me all debit transactions over \$100”.

The associated metadata constraints can additionally include tenant isolation and any schema-value constraints resolved during query analysis.

1.8 Canonical Cache

The canonical cache is tenant-aware. A cache key must incorporate every query attribute that can affect the retrieval result.

A conceptual cache key is therefore

$$K_{\text{cache}}(q) = (K_{\text{canon}}(q), \text{tenant}, F(q)), \quad (21)$$

where $K_{\text{canon}}(q)$ denotes the canonical query identity and $F(q)$ denotes the effective retrieval constraints.

If all retrieval-affecting constraints are already incorporated into $K_{\text{canon}}(q)$, they need not be represented separately.

On a cache hit,

$$K_{\text{cache}}(q) \in \mathcal{C}_{\text{cache}} \Rightarrow \text{retrieval layer is skipped.} \quad (22)$$

On a cache miss,

$$K_{\text{cache}}(q) \notin \mathcal{C}_{\text{cache}} \Rightarrow \text{retrieve} \rightarrow \text{store} \rightarrow \text{return.} \quad (23)$$

The implementation maintains an explicit execution counter to validate that a cache hit does not invoke the retrieval layer again.

The concrete execution-level guarantee is therefore

$$\boxed{\text{same cache key} \Rightarrow \text{same stored retrieval result}} \quad (24)$$

for an unchanged cached entry.
This does not imply

$$\text{semantic equivalence} \Rightarrow \text{same cache key}. \quad (25)$$

That implication remains dependent on the canonicalization mechanism.

Furthermore, cached retrieval results become stale if the underlying knowledge base changes and the cache entry is not invalidated or given an appropriate expiration policy. Consequently, cache correctness in a production system requires an explicit invalidation or versioning strategy.

1.9 Knowledge Base Indexing

Each knowledge-base document is represented as

$$d_i = (\text{id}_i, \text{text}_i, \text{metadata}_i). \quad (26)$$

For each document, the system computes a dense embedding:

$$\mathbf{v}_i = f(\text{text}_i). \quad (27)$$

The resulting vector is inserted into the dense HNSW index, while the document text, identifier, and metadata are maintained separately.

Conceptually, the knowledge base therefore consists of

$$\mathcal{K} = \{I_{\text{HNSW}}, I_{\text{BM25}}, D, M\}, \quad (28)$$

where

- I_{HNSW} is the dense ANN index,
- I_{BM25} is the sparse retrieval structure,
- D is the document store, and
- M contains the associated metadata.

1.10 Dense HNSW Retrieval

The dense retrieval component is implemented using FAISS HNSW.

For a query embedding $\hat{\mathbf{v}}_q$, HNSW returns an approximate nearest-neighbor ranking:

$$R_{\text{dense}}(q, k) = \text{HNSWSearch}(I_{\text{HNSW}}, \hat{\mathbf{v}}_q, k). \quad (29)$$

A key property of the implementation is that HNSW supports incremental insertion.

When a new document with embedding $\hat{\mathbf{v}}_{N+1}$ is added, the dense index is updated as

$$I_{\text{HNSW}} \leftarrow I_{\text{HNSW}} + \hat{\mathbf{v}}_{N+1}. \quad (30)$$

The dense index therefore does not need to be reconstructed from the complete document collection after every insertion.

HNSW construction and search parameters, including graph connectivity and search effort, can be tuned to obtain a desired accuracy–latency–memory trade-off.

1.11 Sparse BM25 Retrieval

The sparse retrieval component uses the `rank_bm25` implementation of BM25.

Given a tokenized corpus

$$D = \{d_1, d_2, \dots, d_N\}, \quad (31)$$

BM25 assigns a relevance score based on term frequency, inverse document frequency, and document-length normalization.

A conceptual BM25 score is

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdL}}\right)}, \quad (32)$$

where $f(t, d)$ denotes the frequency of term t in document d , $|d|$ denotes document length, `avgdL` denotes average document length, and k_1 and b are BM25 parameters.

The current implementation constructs the BM25 object from the complete tokenized corpus.

The `rank_bm25` implementation does not provide a true incremental indexing API. Therefore, adding a document does not immediately rebuild the sparse retrieval structure. Instead, document insertion marks the sparse index as dirty:

$$\text{bm25_dirty} \leftarrow \text{True}. \quad (33)$$

The next sparse-search operation detects the dirty state and rebuilds the BM25 object over the complete current corpus:

$$I_{\text{BM25}} \leftarrow \text{BM25Okapi}(d_1, \dots, d_N). \quad (34)$$

Thus, the implementation provides

$$\boxed{\text{deferred full rebuild}} \quad (35)$$

rather than

$$\boxed{\text{incremental BM25 indexing}}. \quad (36)$$

1.12 Why BM25 Rebuilding Matters

The distinction is particularly important during bulk loading.

Suppose documents $d_1, \dots, d_N$ are inserted.

A naive implementation that rebuild the sparse index after every insertion would perform

$$\text{Build}(d_1) + \text{Build}(d_1, d_2) + \dots + \text{Build}(d_1, \dots, d_N). \quad (37)$$

Because each build processes an increasingly large corpus, this can produce substantially more cumulative work than constructing the sparse index once after bulk loading.

The lazy implementation instead performs

$$d_1, \dots, d_N \rightarrow \text{bm25_dirty} \rightarrow \text{Build}(d_1, \dots, d_N). \quad (38)$$

This substantially reduces the cost of bulk loading.

However, deferred rebuilding does not make BM25 incremental. If documents are continuously added and sparse searches occur between updates, each dirty search can trigger another full-corpus rebuild.

Therefore, for a large production corpus with frequent updates, the sparse component should be replaced by an inverted-index search engine that supports incremental document indexing.

1.13 Metadata Filtering

Metadata constraints determine which documents are eligible to contribute to the retrieval result.

For document d_i and metadata filter F , define

$$\text{eligible}(d_i, F) = \begin{cases} 1, & \text{if all required metadata constraints match,} \\ 0, & \text{otherwise.} \end{cases} \quad (39)$$

For example, a query may produce

$$F = \{\text{tenant} = \text{acme}, \text{transaction_type} = \text{DEBIT}\}. \quad (40)$$

Only documents satisfying the effective metadata constraints should be included in the final candidate set.

If the current implementation applies metadata conditions after the dense and sparse candidate-generation stages, this should be described as *post-filtering*. It should not be described as indexed pre-filtering.

For a production system, indexed metadata filtering is preferable when the constraints can substantially reduce the candidate search space.

Tenant isolation is particularly important because tenant information must also participate in cache isolation:

$$K_{\text{cache}} = (K_{\text{canon}}, \text{tenant}, F). \quad (41)$$

Thus, retrieval results for one tenant must not be reused for another tenant merely because their natural-language queries are identical.

1.14 Hybrid Retrieval

Dense and sparse retrieval provide complementary retrieval signals.

The dense retriever produces a ranked list

$$R_{\text{dense}}(q, k) = \text{DenseSearch}(q, k), \quad (42)$$

while the sparse retriever produces

$$R_{\text{sparse}}(q, k) = \text{BM25Search}(q, k). \quad (43)$$

The implementation retrieves an expanded candidate set from each retriever before rank fusion.

Dense retrieval is particularly useful for semantic similarity and paraphrastic matches. Sparse BM25 retrieval is useful for lexical matches, terminology, identifiers, and other exact or near-exact signals.

The two retrieval mechanisms are therefore combined rather than selecting a single retrieval mechanism exclusively.

1.15 Reciprocal Rank Fusion

The system combines the dense and sparse rankings using Reciprocal Rank Fusion.

Let $\mathcal{R}$ denote the set of retrieval rankings contributing to fusion. For document d , the RRF score is

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k_{\text{RRF}} + \text{rank}_r(d)}, \quad (44)$$

where $\text{rank}_r(d)$ is the one-based rank of document d in ranking r , and k_{RRF} is the RRF constant.

In the current implementation,

$$k_{\text{RRF}} = 60. \quad (45)$$

Documents appearing highly in multiple retrieval rankings therefore receive larger fused scores.

Documents that do not appear in a particular ranking contribute no term from that ranking.

The fused ranking is subsequently truncated to form the candidate set passed to the reranking stage.

1.16 Relevance Reranking

The hybrid retrieval stage is primarily optimized for candidate recall. A smaller candidate set is subsequently passed to a more expensive relevance model.

For each candidate document d_i , the reranker evaluates the query and document jointly:

$$s_{\text{rerank}}(q, d_i) = g(q, d_i), \quad (46)$$

where $g(\cdot, \cdot)$ denotes the selected relevance model.

This differs from independent bi-encoder similarity because the query and candidate passage are evaluated jointly by the relevance model.

The current implementation provides several reranking backends:

- Anthropic-based LLM reranking,
- Cohere reranking,
- GitHub Models API reranking, and
- GitHub Models CLI reranking.

The configured default in the main pipeline is

$$\text{RERANKER_BACKEND} = \text{gh_models_cli}. \quad (47)$$

The reranker receives the query and candidate passages and produces a relevance score for each candidate. The highest-scoring candidates are then retained as top evidence.

The external model call therefore constitutes a genuine relevance reranking stage rather than the earlier local heuristic based on a weighted combination of embedding similarity and keyword overlap.

1.17 Reranker Reliability and Deployment

The reranker is an external dependency and therefore introduces operational considerations that are distinct from the retrieval algorithm itself.

These considerations include:

- authentication,
- API or CLI availability,
- network latency,
- rate limits,

- service availability,
- inference cost, and
- response-format validation.

The GitHub Models CLI backend delegates authentication to the local GitHub CLI authentication mechanism rather than requiring an API token to be embedded directly in the application code.

Consequently, the reference implementation separates the retrieval pipeline from the credentials and authentication mechanism used by a particular deployment.

A production implementation should additionally define timeout, retry, failure, and fallback behavior for unavailable reranking services.

1.18 ANN Recall Validation

The implementation contains a brute-force recall monitor for validating the quality of the HNSW approximation.

For a sampled query q , exact dense similarity is computed against every document:

$$s_i = \hat{\mathbf{v}}_q^\top \hat{\mathbf{v}}_i, \quad i = 1, \dots, N. \quad (48)$$

The exact top- k set is

$$T_k^{\text{exact}} = \text{TopK}(s_1, \dots, s_N). \quad (49)$$

The HNSW index independently produces an approximate top- k set

$$T_k^{\text{ANN}}. \quad (50)$$

Recall at k is measured as

$$\text{Recall @}k = \frac{|T_k^{\text{exact}} \cap T_k^{\text{ANN}}|}{k}. \quad (51)$$

This brute-force calculation is explicitly a validation mechanism. It is not part of the normal production retrieval path because exact similarity computation requires evaluating the query against the entire corpus.

For dense vector retrieval, this introduces a per-query cost that grows linearly with the number of stored vectors:

$$O(N). \quad (52)$$

The production retrieval path therefore uses

$$\boxed{\text{HNSW ANN retrieval}} \quad (53)$$

whereas the validation path uses

$$\boxed{\text{brute-force exact retrieval}}. \quad (54)$$

The exact-retrieval implementation should therefore be treated as an offline or sampled evaluation mechanism rather than a production retrieval strategy.

1.19 Indexing and Query-Time Workflows

The system has two distinct operational workflows: document ingestion and query execution.

1.19.1 Document Ingestion Workflow

The document ingestion process is

Document $\rightarrow$ Embedding $\rightarrow$ HNSW Insertion $\rightarrow$ Document Store Update $\rightarrow$ BM25 Dirty Flag. (55)

The dense index is updated immediately.

The BM25 structure is not reconstructed immediately. Instead, the next sparse retrieval operation detects the dirty state and performs the deferred full rebuild.

1.19.2 Query Execution Workflow

The query workflow is

Query $\rightarrow$ Embedding
 $\rightarrow$ Schema Resolution + Clustering
 $\rightarrow$ Canonical Query
 $\rightarrow$ Cache Lookup
 $\rightarrow$ Dense HNSW + BM25 (56)
 $\rightarrow$ RRF
 $\rightarrow$ Candidate Set
 $\rightarrow$ Reranking
 $\rightarrow$ Top Evidence.

The retrieval branch is executed only on a cache miss.

1.20 Guarantees and Non-Guarantees

The architecture intentionally distinguishes implementation guarantees from empirical expectations.

1.20.1 Guarantees

Under a fixed embedding model, registry state, knowledge-base state, and configuration, the following properties hold:

1. **Deterministic query analysis.** Identical query input produces identical query-analysis output when the embedding function and query-analysis implementation are deterministic.
2. **Shared representation within a cluster.** Queries assigned to the same cluster use the same cluster representative for canonical retrieval representation, provided the representative is unchanged.
3. **Cache bypass on a valid cache hit.** A cache hit for the same cache key returns the stored retrieval result without invoking the retrieval layer.
4. **Tenant-aware cache isolation.** Tenant information is included in the cache identity, preventing retrieval-result reuse across tenants when their tenant values differ.
5. **Incremental dense insertion.** New dense document vectors can be inserted into the HNSW index without reconstructing the entire dense index.

These are implementation-level properties and should not be confused with guarantees about retrieval relevance.

1.20.2 Non-Guarantees

The following claims should not be made:

1. Arbitrary semantic paraphrases are guaranteed to produce the same canonical cluster.
2. Cluster assignment is independent of query arrival order.
3. `rank_bm25` provides true incremental BM25 indexing.
4. The brute-force recall monitor is a production-scale retrieval mechanism.
5. A semantically equivalent query is guaranteed to produce the same retrieval result unless it resolves to the same effective cache key and the relevant underlying knowledge-base and retrieval state remain unchanged.
6. Dense ANN retrieval guarantees the exact nearest-neighbor ranking.
7. RRF guarantees improved relevance relative to either individual retriever.
8. The external reranker guarantees correct relevance judgments.

1.21 Scalability Characteristics

The architecture contains components with favorable scaling properties as well as components that are primarily intended for development, experimentation, or moderate-scale deployments.

Table 1: Scalability characteristics of the reference implementation.

Component	Current implementation	Production consideration
Dense retrieval	FAISS HNSW with incremental insertion	Suitable as an ANN foundation for large-scale retrieval, subject to tuning of graph parameters, memory usage, and search effort.
Sparse retrieval	<code>rank_bm25</code> with deferred full rebuild	Replace with a production inverted-index engine that supports incremental indexing for large or frequently changing corpora.
ANN validation	Brute-force comparison against all stored vectors	Use for offline or sampled recall evaluation rather than normal production retrieval.
Query clustering	In-memory clustering with running centroids	Persist registry state and define concurrency semantics for multi-instance deployments.
Canonical cache	In-memory Python dictionary	Use a shared or persistent cache for horizontally scaled deployments and define expiration or invalidation semantics.
Metadata filtering	Application-level filtering of stored metadata	Use indexed metadata filtering where possible to reduce candidate generation and improve tenant isolation.
Reranking	External LLM or reranking service	Account for latency, cost, authentication, availability, rate limits, timeouts, and fallback behavior.

1.22 Recommended Production Evolution

The current implementation provides a functional reference architecture for evaluating the complete retrieval pipeline. A production deployment should preserve the logical architecture while replacing or extending selected reference components.

The recommended evolution is:

1. Retain HNSW or another production-grade ANN implementation for dense retrieval, with parameters selected using empirical recall and latency measurements.
2. Replace `rank_bm25` with a production inverted-index engine that supports incremental document indexing and efficient lexical search.
3. Move the canonical cache from an in-process dictionary to a shared cache when the application is deployed across multiple instances.
4. Persist the canonical-cluster registry and define concurrency semantics for simultaneous query arrivals.
5. Introduce indexed metadata filtering so that tenant and schema constraints can reduce the retrieval search space before expensive ranking operations.
6. Retain brute-force exact retrieval as an offline or sampled recall-evaluation mechanism rather than as a production retrieval path.
7. Evaluate the external reranker using relevance, latency, cost, availability, and failure-rate measurements before selecting the deployment backend.
8. Replace the development embedding implementation with a production-grade sentence-embedding model or service appropriate for the target domain.
9. Introduce explicit cache invalidation or knowledge-base versioning so that cached retrieval results cannot silently outlive the document state on which they were computed.
10. Define observability metrics for retrieval recall, RRF candidate overlap, reranker latency, cache hit rate, BM25 rebuild frequency, and end-to-end query latency.

1.23 Overall System Interpretation

The architecture should therefore be interpreted as a *scalable retrieval design with a reference implementation*, rather than as a claim that every component is already production-scale.

Its central design properties are

$$\begin{aligned}
 &\text{data-derived schema resolution} \\
 &\quad + \text{dynamic query clustering} \\
 &\quad + \text{canonical-query reuse} \\
 &\quad + \text{incremental dense ANN} \\
 &\quad + \text{sparse lexical retrieval} \\
 &\quad + \text{rank fusion} \\
 &\quad + \text{external relevance reranking.}
 \end{aligned} \tag{57}$$

The system consequently combines semantic and lexical retrieval while allowing its query vocabulary to evolve from observed traffic rather than requiring a complete manually authored domain-alias registry in advance.

At the same time, the architecture explicitly recognizes the boundaries of its guarantees. Dynamic clustering is state-dependent and its sequential membership decisions can depend on query arrival order. BM25 is currently rebuilt over the complete corpus when marked dirty, and brute-force retrieval is used only as a validation reference. Metadata filtering is an application-level concern in the reference implementation rather than an inherently indexed retrieval operation.

These properties should be treated as explicit implementation characteristics rather than hidden assumptions.

The resulting architecture therefore separates three distinct concerns:

1. **retrieval efficiency**, provided primarily by dense ANN and sparse lexical retrieval;
2. **retrieval effectiveness**, improved through hybrid retrieval, rank fusion, and relevance reranking; and
3. **operational scalability**, which depends on replacing or extending selected reference components such as the in-memory cache, deferred BM25 implementation, and in-memory canonical registry.

This separation provides a clearer basis for evaluating the system experimentally and for identifying which observed improvements arise from semantic retrieval, lexical retrieval, caching, canonicalization, or reranking.